

SIT 385 NETWORK SYSTEMS ADMINISTRATION

Week 8	Name services	• Introduction, Domain Name Services (DNS), DNS daemons, clients, servers, reverse DNS, • Windows Internet Name Service (WINS).
-----------	---------------	---

What is DNS? Domain Name System, DNS Server, and IP Address Concepts Explained

Introduction

By the end of this article, you should have a better understanding of:

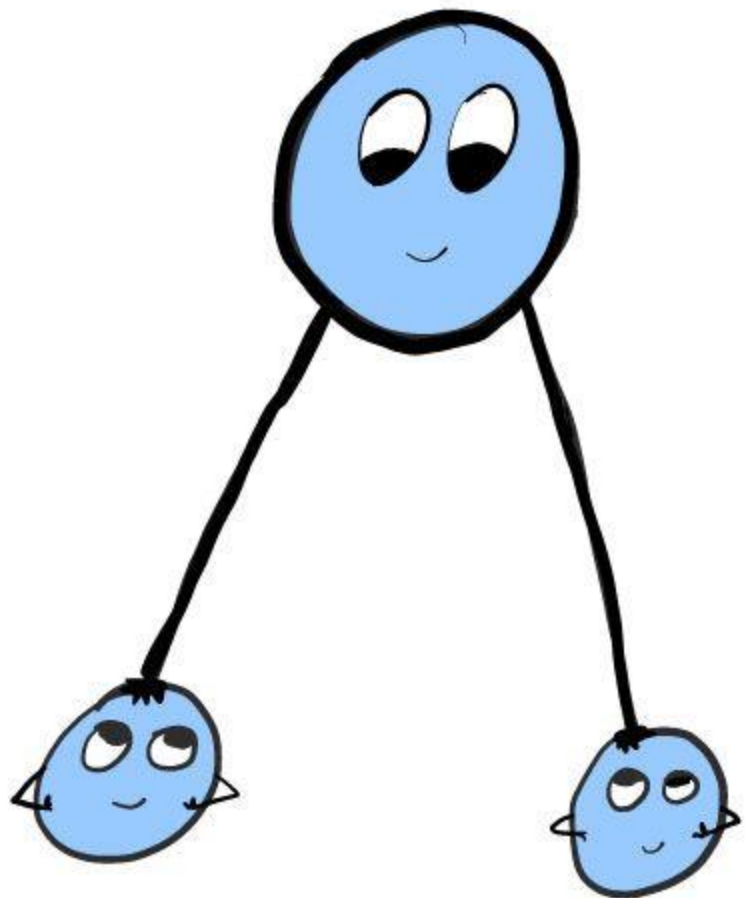
1. What DNS is and what it does
2. What DNS servers do
3. How Internet Protocol (IP) Addresses work in the context of DNS

Important concepts

There are some essential mental models to be familiar with when learning about DNS, DNS servers, and IP addresses. Going over these concepts now, before starting to learn about DNS, will

- help make sense of all the different terms used to describe behavior that fits into these models, and
 - aid in memory retention.
 - Mental models give you a frame of reference when things get a little weird and unfamiliar.
 - So let's lay the groundwork.
-
- Query and response. This is when Thing 1 asks Thing 2 for something, and Thing 2 responds to that request. Like this:
 - Query and Response
 - Parent-child node relationships and graphs that look like this (only more complicated).

Parent node



Child node

Child node

Tree graph

- Messages. It's not a query and response because there is no response. In the world of DNS, the formatting and content of messages vary according to usage.
- Message
- Client-server relationship. In simplest terms, a server is a software or hardware device that provides functionality for other software or hardware devices, called "clients."

Prepare for a lot of talk about servers. As it turns out, there's a whole lot of servers that go into this thing we call DNS, and how we, as humans, use it when we connect to the Internet.

Client-server relationship

What is DNS?

The Domain Name System (DNS) maps human-readable domain names (in URLs or in email address) to IP addresses. For example, DNS translates and maps the domain freecodecamp.org to the IP address 104.26.2.33.

To help you fully understand this description, this section details:

- Historical context for the development of DNS - what problems were it and IP addresses solving?
- Domain names
- IP addresses

Historical Context

In 1966, the Advanced Research Projects Agency (ARPA), a US government agency, founded a computer network called ARPAnet. In simple terms, think of ARPAnet as the first iteration of what we now know today as the Internet.

The main goals of ARPAnet included

“(1) providing reliable communication even in the event of a partial equipment or network failure, (2) being able to connect to different types of computers and operating systems and (3) being a cooperative effort rather than a monopoly controlled by a single corporation. In order to provide reliable communication in the face of equipment failure, ARPANET was designed so that no one point or link was more critical than any other. This was accompanied by the building of redundant routes and the use of on-the-fly rerouting of data if any part of the network failed.”

The Problems

DNS and TCP/IP were critical in solving two issues with ARPAnet:

For ARPAnet, there was a single location (a file called HOSTS.TXT) that contained all name-to-address mapping for every host on the network.

“HOSTS.TXT was maintained by SRI’s *Network Information Center* (dubbed “the NIC”) and distributed from a single host, SRI-NIC.[*] ARPAnet administrators typically emailed their changes to the NIC, and periodically FTP’ed to SRI-NIC and grabbed the current HOSTS.TXT file. Their changes were compiled into a new *HOSTS.TXT* file once or twice a week.”

There were three challenges with this set-up:

1. Traffic and load - distributing the file was becoming too much for the responsible host to handle.
2. Name collisions - each host had to have a unique name, and there was no centralized authority that prevented network users from adding a host with a conflicting (non-unique) name, thereby “breaking the whole scheme.”
3. Consistency - the act of updating the file and ensuring all hosts had the most updated version became impossible or at least very difficult.

In essence, HOSTS.TXT was a single point of failure, so the entire process here didn’t scale well past a certain number of hosts. ARPAnet needed a *decentralized* and scalable solution. DNS was it. [Source](#)

Host-to-host communication within the same network wasn’t reliable enough. TCP/IP helped solve this issue.

1. Transmission Control Protocol (TCP) provides quality assurance measures for the process of turning messages (between hosts) into packets. The TCP protocol is connection-oriented, which means a connection between source host and destination host must be established.
2. Internet Protocol (IP) defines how messages (packets) are carried between source host and destination host. An IP address is a unique identifier for a specific path that leads to a host on a network.
3. TCP and IP work closely together, which is why they’re usually referenced like “TCP/IP.”
4. While I won’t dive into it in this article, both TCP and User Datagram Protocol (UDP) are used in the data transport layer of DNS. UDP is faster, much less reliable, and doesn’t require connections; TCP is slower, much more reliable, but needs connections. They are used as needed and appropriate in DNS; needless to say, the inclusion of TCP in ARPAnet was a valuable addition to the data transport layer.

By the early 1980s, DNS and TCP/IP (and therefore, IP addresses) were standard operating procedures for the ARPAnet.

This history is *very* abridged. If you'd like to learn more about these topics, please reference the Resources section at the end of this article.

Now that we have some historical context, let's move on to learning more about domain names and IP addresses.

Domain Names

In the context of DNS, a domain name provides a user-friendly way to point to non-local resources. This could be a website, a mail system, print server, or any other server that is available on the Internet. A domain name can be more than just a website!

“The goal of domain names is to provide a mechanism for naming resources in such a way that the names are usable in different hosts, networks, protocol families, internets, and administrative organizations.”

A domain name is much easier to remember and enter into a terminal or Internet browser, than an IP address.

A domain name is part of a Uniform Resource Locator (URL), but the terms are *not synonymous*. A URL is the complete web address of a resource, while the domain name is the name of a website and is a sub-component of every URL.

While there are technical distinctions between URLs and domain names, web browsers usually treat them the same way, so you'll get to the website if you type in the complete web address, or just the domain name.

Top Level Domains and Second Level Domains

There are two parts to any given domain: top-level domain (TLD) and second-level domain (SLD). The parts of a domain name become more specific when moving from the right (end) to the left (beginning).

This can be confusing at first. For example, let's look at “freecodecamp.org”

- URL: <https://www.freecodecamp.org>
- Domain name: freecodecamp.com
- TLD: org
- SLD: freecodecamp

In the early days of ARPAnet, there were a limited number of TLDs available, including com, edu, gov, org, arpa, mil, and 2-letter country code domains. These TLDs were initially reserved for institutions participating in the ARPAnet, but some later became available on commercial markets.

Today, there is a comparative wealth of available TLDs, including net, aero, biz, coop, info, museum, name, and others.

Second-level domains are the domains that are available for individual purchase through domain registrars (for example, Namecheap).

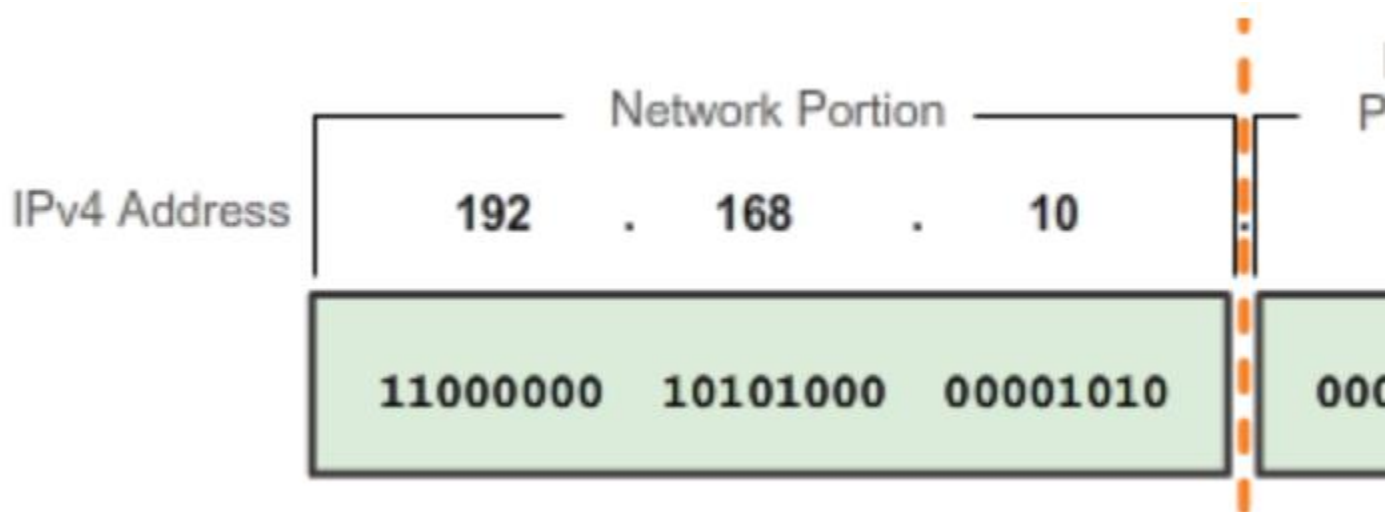
IP Addresses

While IP addresses are related to DNS in their function, the Internet Protocol itself is technically separate from DNS. I've already provided historical context for this distinction, so now I'll explain how IP addresses function.

An IP address, as previously mentioned, is a unique identifier for a specific path that leads to a host on a network. I'd like to reference the analogy of a phone number and a phone: a phone number doesn't represent the phone itself, it's just a way to reach the person with the phone.

This analogy is reasonably appropriate (at least, on a surface level), with IP addresses. An IP address represents an endpoint, but it isn't the endpoint itself. IP assignments can be fixed (permanent) or dynamic (flexible and may be reassigned).

Like a domain name, the organization of IP addresses follows a hierarchical structure. Unlike domain names, IP addresses get more specific going left-to-right. This is an IPv4 example below:



This diagram shows that 129.144 is the network part and 50.56 is the host part of an IPv4 address.

- Network: the unique number assigned to your network
- Host: identifies the host (machine) on the network

If greater specificity is needed, network administrators can subnet the address space and delegate additional numbers.

How many IP addresses are there?

IPv4 was the very first iteration of IP that ARPAnet used in production. Deployed in the early 80s, it's still the most prevalent IP version. It's a 32-bit scheme, and can therefore support slightly over 4 billion addresses.

But wait, is that enough? Nope.

IPv6 has a 128-bit scheme, which allows it to support 340 undecillion addresses. It also offers performance improvements on IPv4.

Example IPv4 address:

- 104.26.2.33 (freeCodeCamp)

Example IPv6 address:

- 2001:db8:a0b:12f0::1 (the compressed format and not pointing to freeCodeCamp)

How does the Domain Name System work?

So, we've learned about domain names! We've learned about IP addresses! Now how do they relate to the Domain Name System?

First of all, they fit into the namespace.

The Domain Namespace

As implied by the language "top" level domain and "second" level domain, the namespace is based on a hierarchy

"...with the hierarchy roughly corresponding to organizational structure, and names using "." as the character to mark the boundary between hierarchy levels." [Source](#).

This tree graph, with the root at the top, best illustrates the structure:

The Namespace

Let's break this down, starting at the top.

The top of this graph, noted with a "." is called the "root."

“The authoritative name servers that serve the DNS root zone, commonly known as the “root servers”, are a network of hundreds of servers in many countries around the world. They are configured in the DNS root zone as 13 named authorities.”

The root domain has a zero-length label.

From here-on down, each node (dot) in the graph has a unique label up to 63 characters long.

The first level down from the root are the TLDs: the com, org, edu, and gov. Please note that this graph does not contain a full list of TLDs.

Below TLDs are the SLDs, the second-level domains. The children of each node are called “subdomains,” which are still considered part of the parent domain. For example, in freecodecamp.org, freecodecamp (the SLD) is a subdomain of org (the TLD).

Depending on the hierarchy of the website, there may be third-, fourth, fifth- level domains. For example, in hypothetical-subdomain.freecodecamp.org, hypothetical-subdomain is the third-level domain, and the subdomain of freecodecamp. So on and so forth, at least up to 127 levels, which is the maximum allowed by DNS.

Who manages the namespace?

Wouldn’t it be nuts to try to have one person or organization administer everything? Yes, it would. Especially because one of the chief design goals of DNS was to promote distributed, decentralized management of the system at large.

Each domain (or subdomain) in the domain namespace is or is part of a *zone*, “an autonomously administered piece of the namespace.” So, the namespace is broken into zones. Responsibility for those zones is managed through delegation and administration.

The process of assigning the responsibility of subdomains to other entities is called *delegation*. For example, the Public Interest Registry administers the domain name org, and has since 2003. Public Interest Registry may delegate responsibility to other parties to manage subdomains of org, say freecodecamp. And then whoever administers freecodecamp may assign responsibility for the subdomains of freecodecamp (for example, hypothetical-subdomain.freecodecamp.com) to another party.

If someone (an organization, team, or individual) administers a zone, what they’re doing is administering the nameserver that is responsible for the zone.

This brings us into one of the most foundational concepts in the Domain Name System.

Domain Name Servers

“The programs that store information about the domain namespace are called nameservers.”

At this point is where thinking about a client-server relationship, at least initially, is useful.

Domain nameservers are the “server” side of the client-server relationship. Nameservers may load one, hundreds, or even thousands of zones, but they never load the entire namespace. Once a nameserver has loaded the totality of a zone, it is said to be *authoritative* for that zone.

To understand why nameservers function the way they do, it’s useful to understand the “client” part of the relationship.

Resolvers

In DNS, the client (the requester of information) is called the “resolver,” which may seem backward at first. Wouldn’t the server that is resolving the request be called the “resolver?” I thought so, too, but it’s not. Best to just memorize that and move on.

Resolvers are typically included, de facto, in most operating systems, so the applications installed on the OS don’t have to figure out how to make low-level DNS queries.

DNS queries and their responses are types of DNS messages, and have their own data transport protocol (usually UDP). Resolvers are responsible for helping applications installed on the OS translate requests for DNS-related data into DNS queries.

In sum, resolvers are responsible for packaging and sending off requests for data. Once the resolver receives the response (if at all), it passes that back to the original requesting application in a format consumable to the requesting application.

Back to Nameservers

Now that we are a bit more familiar with the client-side of the relationship, we need to understand how domain nameservers respond to resolver queries.

Nameservers respond to DNS queries through *resolution*. Resolution is the process by which nameservers find datafiles in the namespace. Depending on the type of query, nameservers respond differently to different queries, but the end goal is resolution.

Query Types

Type of query? Yes, there are multiple types of DNS queries. But first, what’s usually in a DNS query? It’s a request for information, specifically for the IP address associated with a domain name.

- *Recursive*: recursive queries allow the query to be referred on to multiple nameservers to be resolved. If the first queried nameserver doesn’t have the desired data, then that

nameserver sends the query along to the most appropriate next nameserver, until the nameserver with the desired datafiles is found and sends a response to the resolver.

- *Iterative*: iterative queries require the queried nameserver to respond either with the desired data or with an error. The response may contain the IP address of the most appropriate nameserver to send the request to next; the resolver may then send another request to that, more appropriate, nameserver.

In case you need it, you can also query for the domain name, if all you have is the IP address. This is called a reverse DNS lookup.

Once the query reaches a nameserver that contains the desired datafiles, then the query can be resolved. Nameservers have a number of datafiles associated with them, all or some of which may be used to resolve the query.

Resource Records (RRs)

These are the datafiles in the domain namespace. These datafiles have specific formats and **contents**.

The most common RRs:

- A Record: if you haven't heard of any other RRs except for this one, that would make sense. It's likely the best-known RR and contains the IP address of the given domain.
- CNAME Record: if you haven't heard of any other RRs except for this one and the A record, that would also make sense. The "C" stands for "canonical", and is used instead of an A record, to assign an alias to a domain.
- SOA Record: this record contains administrative information about the one, including the email address of the administrator. Hint: if you administer a zone, make sure there's a valid email address here, so folks can get in touch with you if needed.

Caching and Time to Live (TTL)

When the local nameserver receives a response from a query, it caches that data (stores it in memory), so next time it receives the same query, it can just answer the query directly rather than go through the original, longer resolution process.

But once this information is cached, it is both static and isolated, and is therefore at risk of becoming out of date. Therefore, resource records all have what is called a *time to live* (TTL) value, which dictates how long that data can be cached. When that time runs out (reaches zero), the nameserver deletes the record.

Important note: TTL doesn't apply to the name servers that are authoritative for the zone that contains the resource record. It just applies to the nameserver that cached that resource record.

Conclusion

At this point, you should understand what DNS is and what a nameserver is, as well as be familiar with technical concepts relating to IP addresses.

Many books have been written about and dive deeper into the fascinating world of DNS, and there is so much more to learn. The topics that were not included in this article but are either part of DNS or very related include:

- Nameserver implementations
- Forwarding
- (More about) node labels
- Primary and secondary nameserver relationships
- Retransmission algorithms
- Load balancing
- Plus, other more general topics about the how the Internet functions, like:
- World Wide Web
- HTTP
- FTP
- Communication protocol layers: link layer, IP layer, transport layer, Internet layer, etc.

For those of you who are still reading and want to learn more about DNS, I first and foremost recommend “DNS and BIND, 5th Ed.”, written by Cricket Liu and published by O’Reilly Media. It’s invaluable.

I also encourage everyone to poke around in the original Request for Comments (RFCs) linked below. Not only are there points for reading primary sources, but they’re also exceptionally well-organized and comprehensible documents, which is why I quoted them in this article.

WINS Overview

The Windows Internet Naming Service (WINS) provides NetBIOS name resolution for clients on a network. NetBIOS names are used by legacy clients such as Windows NT 4.0 and Windows 95. A WINS server stores a database containing NetBIOS name to IP address mappings. A Windows Server 2003 machine can be configured as a WINS Server. Even though a native Windows 2003 environment can do without WINS, WINS may still be required by some legacy applications. Most modern networks contain a broad range of clients and applications, therefore WINS remains an important service. WINS servers can also share their databases with other

WINS servers (Replication). This allows clients on remote subnets to communicate using NetBIOS names.

WINS Name Resolution

Every Microsoft computer is configured to use one of the 4 different node types. A Node Type tells the computer how to resolve a name. Microsoft recommends that you configure a computer to use H-Node resolution.

- Broadcast (B-Node).
- Peer (P-Node).
- Mixed (M-Node).
- Hybrid (H-Node).

Broadcast (B-node) – B-node uses only a broadcast to resolve NetBIOS names. Clients configured with B-node will never use WINS. B-node is often used when there are no name servers available.

Peer (P-node) – P-node uses WINS only. It will never use a broadcast, even if the WINS server cannot resolve the name. It is used to reduce traffic in large networks.

Mixed (M-node) – M-node uses a combination of broadcast and WINS. Clients configured to use M-node first use a broadcast (B-node) to resolve; if that is unsuccessful, they query the configured WINS server (P-node).

Hybrid (H-node) – H-node uses a combination of broadcast and WINS. Clients configured to use H-node first query the WINS server (P-node); if that is unsuccessful, they try broadcast (B-node) to resolve.

When a H-node client configured to use WINS needs to resolve a hostname, it goes through the following stages:

- Client checks its NetBIOS name cache.
- Client queries the WINS server.
- Client sends a broadcast.
- Client checks its LMHOSTS file.
- Client checks its HOSTS files.
- Client queries a DNS Server.

To find out which node type a client is using. Use the `ipconfig /all` command from a command prompt.

When a Windows client computer enters the network, it registers its name with a WINS Server so that other client machines can resolve its name to an IP address. By default a client is allowed its name for six days before it needs to renew it. This allows the name to be freed up when not in use.

When a client is shut down, its name is marked as released. The name still belongs to the client but the name is not in use. By default, the name is marked as extinct for four days before being removed. In a very dynamic environment with computers frequently entering and exiting the network, the renewal interval and extinction level can be lowered.